

ArduPlane 代码架构大纲

本文梳理了 ArduPlane (含 QuadPlane/VTOL) 的代码结构, 便于后续定位问题。

一、目录文件分类 (80个文件)

```
1 ArduPlane/
2 |— 核心文件 (5)
3 |   |— Plane.h / Plane.cpp          ← 主类定义 / 调度器+主循环
4 |   |— mode.h / mode.cpp            ← 飞行模式基类+20+派生类
5 |   |— defines.h                    ← 所有关键常量/枚举
6 |   |— Parameters.h / Parameters.cpp ← 参数表(G1+G2两组)
7 |   |— config.h                      ← 编译配置
8 |
9 |— 子系统文件 (15)
10 |   |— system.cpp                  ← init_ardupilot() + set_mode()
11 |   |— Attitude.cpp               ← stabilize() + PID控制(滚转/俯仰/偏航)
12 |   |— navigation.cpp              ← navigate() + 航点追踪 + Loiter
13 |   |— servos.cpp                  ← set_servos() + 舵机输出+混控
14 |   |— radio.cpp                   ← read_radio() + RC输入+失控检测
15 |   |— RC_Channel_Plane.cpp        ← 辅助开关(AUX function)分发
16 |   |— commands.cpp / commands_logic.cpp ← 航点管理+任务命令
17 |   |— events.cpp                  ← 失控处理(failsafe short/long/battery)
18 |   |— sensors.cpp                 ← 测距仪读取
19 |   |— Log.cpp                     ← 数据日志(CTUN/NTUN/QTUN/AETR...)
20 |   |— tuning.cpp                  ← 遥控器空中调参
21 |   |— GCS_Plane.cpp / GCS_MAVLink_Plane.cpp ← 地面站通信
22 |   |— ArduPlane.cpp               ← (主要是废弃逻辑, 已迁移到各子系统)
23 |
24 |— 飞行模式 (25个 mode_*.cpp)
25 |   |— mode_manual.cpp / mode_stabilize.cpp / mode_training.cpp /
mode_acro.cpp
26 |   |— mode_fbwa.cpp / mode_fbwb.cpp / mode_cruise.cpp
27 |   |— mode_auto.cpp / mode_rtl.cpp / mode_guided.cpp / mode_takeoff.cpp
28 |   |— mode_circle.cpp / mode_loiter.cpp / mode_autotune.cpp
29 |   |— mode_thermal.cpp / mode_autoland.cpp / mode_avoidADSB.cpp
30 |   |— mode_qstabilize.cpp / mode_qhover.cpp / mode_qloiter.cpp
31 |   |— mode_qland.cpp / mode_qrtl.cpp / mode_qacro.cpp / mode_qautotune.cpp
32 |   |— mode_LoiterAltQLand.cpp     ← 继承 ModeLoiter
33 |
34 |— VTOL/过渡 (6)
35 |   |— quadplane.h / quadplane.cpp ← QuadPlane 主类
36 |   |— transition.h/.cpp           ← SLT_Transition (独立升力推力)
37 |   |— tailsitter.h/.cpp           ← Tailsitter + Tailsitter_Transition
38 |   |— tiltrotor.h/.cpp            ← Tiltrotor + Tiltrotor_Transition
39 |   |— VTOL_Assist.h/.cpp          ← 固定翼模式下的VTOL辅助
40 |   |— qautotune.h/.cpp            ← 四旋翼自动调参
41 |
42 |— 专项功能
43 |   |— soaring.cpp                 ← 热气流滑翔
44 |   |— pullup.h/.cpp               ← 自动拉升/避障
45 |   |— systemid.h/.cpp             ← 系统辨识
```

```

46 | | └─ AP_Arming_Plane.h/.cpp ← 上锁/解锁逻辑
47 | | └─ fence.cpp ← 电子围栏
48 | | └─ parachute.cpp ← 降落伞
49 | | └─ motor_test.cpp ← 电机测试
50 | | └─ is_flying.cpp ← 飞行状态检测
51 | | └─ ekf_check.cpp ← EKF健康检查
52 | | └─ reverse_thrust.cpp ← 反推
53 | | └─ avoidance_adsb.cpp ← ADSB规避

```

二、核心类继承体系

```

1 AP_Vehicle (libraries/AP_Vehicle)
2   └─ Plane (ArduPlane/Plane.h) ← 核心类, 全局单例 plane
3   |
4   | └─ has-a → QuadPlane ← VTOL管理 (条件编译
HAL_QUADPLANE_ENABLED)
5   | | └─ has-a → Transition* ← 基类指针
6   | | | └─ SLT_Transition (独立升力推力)
7   | | | └─ Tailsitter_Transition(尾座式)
8   | | | └─ Tiltrotor_Transition (倾转旋翼)
9   | | └─ has-a → Tailsitter
10  | | └─ has-a → Tiltrotor
11  | | └─ has-a → VTOL_Assist
12  | | └─ has-a → AP_MotorsMulticopter* ← 多旋翼电机混控
13  | | └─ has-a → AC_AttitudeControl_Multi* ← VTOL姿态控制
14  | | └─ has-a → AC_PosControl* ← 位置控制
15  | | └─ has-a → AC_WPNav* ← 航点导航
16  | | └─ has-a → AC_Loiter* ← 悬停导航
17  |
18  | └─ has-a → Mode* (control_mode) ← 当前飞行模式指针
19  | | └─ Mode (基类, mode.h)
20  | | | └─ 固定翼模式: Manual, Stabilize, Training, Acro,
21  | | | | └─ FBWA, FBWB, Cruise, Auto, RTL, Guided,
22  | | | | └─ Loiter, Circle, Takeoff, AutoTune, Thermal,
23  | | | | └─ AutoLand, AvoidADSB
24  | | | └─ VTOL模式: QStabilize, QHover, QLoiter, QLand,
25  | | | | └─ QRTL, QAcro, QAutotune
26  | | | └─ 混合模式: LoiterAltQLand (继承 ModeLoiter)
27  |
28  | └─ has-a → AP_TECS ← 总能量控制系统(高度/速度耦合)
29  | └─ has-a → AP_L1_Control ← L1导航控制器(横向)
30  | └─ has-a → AP_RollController / AP_PitchController /
AP_YawController / AP_SteerController
31  | | └─ has-a → AP_Mission ← 任务系统
32  | | └─ has-a → AP_Landing ← 降落管理
33  | | └─ has-a → AP_Camera / AP_Mount / AP_Parachute / AP_ADSB / AP_Rally
/ AP_Terrain ...

```

三、主循环调度器 (Plane.cpp)

3.1 scheduler_tasks[] 优先级排列

```
1 FAST_TASK (每圈都跑):
2   1. ahrs_update()           ← AHRS姿态更新
3   2. update_control_mode()   ← 检查模式切换, 调用 control_mode->update()
4   3. stabilize()             ← 调用 control_mode->run() → PID稳定
5   4. set_servos()            ← 输出PWM到舵机/电机
6
7 固定频率任务:
8   50Hz: read_radio(), read_rangefinder(), check_short_rc_failsafe()
9   10Hz: navigate(), update_alt() → TECS计算, Log_Write_Fast()
10  5Hz:  is_flying检测, crash检测
11  3Hz:  check_long_failsafe(), gcs_send_heartbeat()
12  1Hz:  one_second_loop() (通道映射, 更新home, ADSB等)
13
14 其他: 罗盘累积(50Hz), GPS更新(50/10Hz), 日志写入, fence检查, ADSB规避等
```

3.2 每圈控制流

```
1 read_radio() → update_control_mode() → stabilize() → set_servos()
2                                     ↑ 10Hz补充: navigate() →
   update_alt()
```

四、飞行模式系统

4.1 Mode 基类关键虚方法

方法	调用频率	用途
<code>enter() / _enter()</code>	模式切换时	初始化模式状态 (重置标志位, 设置目标值)
<code>exit() / _exit()</code>	模式切换时	清理/保存状态
<code>update()</code>	FAST_TASK (每圈)	模式特定的 导航层 逻辑 (计算导航需求)
<code>run()</code>	FAST_TASK (每圈)	模式特定的 稳定层 逻辑 (PID控制)
<code>navigate()</code>	10Hz	航点追踪 (仅 auto/rtl/guided 等有自动导航的模式)

基类查询方法 (模式能力声明):

```
is_vtol_mode(), is_guided_mode(), does_auto_navigation(), does_auto_throttle(),
allows_throttle_nudging(), allows_terrain_disable(), is_landing(), is_taking_off()
```

4.2 所有飞行模式一览

1	Mode::Number 枚举:	
2	0 MANUAL	- 纯手动, 无稳定, 直通舵机
3	1 CIRCLE	- 自动盘旋 (does_auto_navigation + does_auto_throttle)
4	2 STABILIZE	- 姿态稳定 + 飞手输入
5	3 TRAINING	- 训练模式 (限幅角度)
6	4 ACRO	- 特技模式 (角速度控制, 四元数姿态)
7	5 FBWA	- Fly-by-wire A (角度稳定, 飞手控制油门)
8	6 FBWB	- Fly-by-wire B (定高定速)
9	7 CRUISE	- 巡航 (锁定航向, 飞手油门为trim)
10	8 AUTOTUNE	- 自动PID调参
11	10 AUTO	- 自主任务执行 (mission.update)
12	11 RTL	- 返航 (爬升→返航高度→盘旋→进场→降落)
13	12 LOITER	- 当前位置盘旋
14	13 TAKEOFF	- 自主起飞 (有自己的参数组)
15	14 AVOID_ADSB	- ADSB碰撞规避 (is_guided_mode)
16	15 GUIDED	- 地面站/外部控制 (handle_guided_request)
17	16 INITIALISING	- 启动占位 (_enter 返回 false)
18	17 QSTABILIZE	- VTOL姿态稳定 (is_vtol_mode)
19	18 QHOVER	- VTOL悬停 (is_vtol_mode)
20	19 QLOITER	- VTOL位置保持 (is_vtol_mode, 有自己的pos control)
21	20 QLAND	- VTOL降落 (is_vtol_mode, 着陆状态机)
22	21 QRTL	- VTOL返航 (is_vtol_mode, 包含 RTL→LOITER_TO_ALT→进场→降落)
23	22 QAUTOTUNE	- VTOL自动调参 (is_vtol_mode)
24	23 QACRO	- VTOL特技 (is_vtol_mode, 手动油门)
25	24 THERMAL	- 热气流滑翔
26	25 LOITER_ALT_QLAND	- 盘旋+QLand备降 (继承 ModeLoiter)
27	26 AUTOLAND	- 自主降落 (有自己的参数组)

4.3 Mode 基类 run() 默认流程

1	Mode::run() 默认实现:
2	1. 检查 STICK_MIXING 参数 → 决定是否混控飞手杆量
3	2. stabilize_roll() → 滚转PID
4	3. stabilize_pitch() → 俯仰PID
5	4. stabilize_yaw() → 偏航PID (协调转弯 或 航向保持 或 地面转向)

五、QuadPlane/VTOL 子系统

5.1 推力类型 (ThrustType)

1	SLT	← 独立升力推力 (前拉/前推电机 + 多旋翼电机分开)
2	TAILSITTER	← 尾座式 (机身竖立起降)
3	TILTROTOR	← 倾转旋翼 (电机/旋翼可倾转)

5.2 位置控制状态机

```
1 QPOS_NONE → QPOS_APPROACH → QPOS_AIRBRAKE → QPOS_POSITION1 →  
2 QPOS_POSITION2 → QPOS_LAND_DESCEND → QPOS_LAND_FINAL → QPOS_LAND_COMPLETE  
3  
4 特殊:  
5 QPOS_LAND_ABORT - 中止降落, 回到悬停
```

5.3 降落进场阶段 (VTOLApproach::Stage)

```
1 RTL → LOITER_TO_ALT → ENSURE_RADIUS → WAIT_FOR_BREAKOUT → APPROACH_LINE →  
VTOL_LANDING
```

5.4 过渡状态机 (固定翼 ↔ VTOL)

```
1 SLT_Transition:  
2 AIRSPEED_WAIT → TIMER → DONE  
3  
4 Tailsetter_Transition:  
5 ANGLE_WAIT_FW → ANGLE_WAIT_VTOL → DONE  
6  
7 Tiltrotor_Transition:  
8 继承 SLT_Transition, 覆盖 update_yaw_target / show_vtol_view / allow_vfwd
```

5.5 VTOL_Assist (固定翼模式下的四旋翼辅助)

1 Speed Assist	← 空速低于 Q_ASSIST_SPEED 时, 四旋翼电机辅助
2 Angle Assist	← 姿态误差超过 Q_ASSIST_ANGLE 时辅助
3 Altitude Assist	← 检测到掉高时辅助
4 Force Assist	← 遥控器开关强制启用 Q_ASSIST_FORCE

5.6 QuadPlane 关键参数 (var_info)

1 Q_ENABLE	← 主开关 (0:关闭, 1:启用, 2:启用VTOL AUTO)
2 Q_FRAME_CLASS	← 机架类型 (Quad, Hexa, Octa, Tri, Y6...)
3 Q_FRAME_TYPE	← 机架布局 (X, +, H, V...)
4 Q_TRANSITION_MS	← 过渡时间 (达到最小空速后再等多少ms完成过渡)
5 Q_PILOT_SPD_UP/DN	← 飞手控制时的升降速率
6 Q_LAND_FINAL_SPD/ALT	← 降落末段速度和高度
7 Q_ASSIST_SPEED	← 辅助触发空速
8 Q_VFWD_GAIN	← 前向速度保持增益
9 Q_OPTIONS	← 23+ 个位掩码选项 (水平过渡, FW起降, 失控行为等)

六、核心子系统详解

6.1 Attitude.cpp — 姿态稳定

```
1  关键函数：
2  stabilize()                ← FAST_TASK入口，调用 control_mode->run()
3  calc_speed_scaler()        ← 基于空速的动态PID增益缩放
4  stabilize_roll()           ← 滚转PID → 副翼通道
5  stabilize_pitch()          ← 俯仰PID → 升降通道（处理倒飞，VTOL从属，起飞拉杆，降落flare）
6  stabilize_yaw()            ← 偏航三模式：航向保持地面转向 / 角速度地面转向 / 协调转弯
7  calc_nav_roll()            ← 导航需求 → 滚转限制
8  calc_nav_pitch()           ← 导航需求 → 俯仰限制
9  calc_throttle()            ← 油门需求计算
10 apply_load_factor_roll_limits() ← 基于载荷因子的滚转限制（防失速）
```

6.2 navigation.cpp — 导航

```
1  关键函数：
2  navigate()                 ← 10Hz，计算航点距离/方位，调用 control_mode->navigate()
3  update_loiter()            ← Loiter导航（半径+方向 → L1控制器）
4  calc_airspeed_errors()     ← 多源空速目标（FBWB杆量/滑翔/降落/AUTO）+ 地速补偿
5  calc_gndspeed_undershoot() ← 地速不足补偿
6  update_fbwb_speed_height() ← FBWB/CRUISE 高度控制
```

6.3 servos.cpp — 舵机输出

```
1  关键函数：
2  set_servos()               ← FAST_TASK，主输出口
3  set_throttle()             ← 油门限制/电压补偿/抑制逻辑
4  servos_output()            ← 混控输出（elevon, vtail, flaperon, spoiler, twin-engine）
5  throttle_slew_limit()      ← 油门变化率限制
6  suppress_throttle()        ← 判定是否抑制油门（地面检测/降落/降落伞）
7  set_servos_flaps()         ← 襟翼混控
8  dspoiler_update()          ← 差动扰流板
9  servos_auto_trim()         ← 自动微调（监控I项 → 逐步写入trim → 每10秒保存FRAM）
```

6.4 radio.cpp — RC遥控输入

```
1  关键函数：
2  read_radio()               ← 50Hz，读取RC + 失控检测
3  set_control_channels()     ← 1Hz，映射通道（roll/pitch/throttle/rudder）
4  control_failsafe()         ← 失控状态机（FAILSAFE_NONE → FAILSAFE_SHORT）
5  trim_radio()               ← 自动配平（副翼/升降/方向/elevon/vtail）
6  rudder_input()             ← 方向舵输入（含 rudder_only 和 DIRECT_RUDDER_ONLY）
```

6.5 events.cpp — 失控处理

```
1 关键函数：
2  failsafe_short_on_event() ← 短时失控（切 FBWA/FBWB/CIRCLE/QLAND）
3  failsafe_short_off_event() ← 短时失控恢复
4  failsafe_long_on_event() ← 长时失控（切
   RTL/GLIDE/PARACHUTE/AUTO/AUTOLAND）
5  failsafe_long_off_event() ← 长时失控恢复
6  handle_battery_failsafe() ← 电池失控
   (QLand/LoiterAltQLand/Land/RTL/Terminate/Parachute)
```

6.6 system.cpp — 启动与模式切换

```
1 关键函数：
2  init_ardupilot() ← 主初始化（INS/RC/Baro/GPS/Compass/Mission/Logger...）
3  startup_INS() ← 启动AHRS，设置 vehicle_class=FIXED_WING
4  set_mode() ← 中心模式切换（验证→ new_mode.enter() → old_mode.exit()
   → 日志）
5  check_long_failsafe() ← 3Hz，监控RC/GCS心跳
```

6.7 commands.cpp / commands_logic.cpp — 任务系统

```
1  commands.cpp:
2  set_next_WP() ← 加载下一个航点
3  set_guided_WP() ← 设置Guided目标
4  update_home() ← 自动更新home位置
5
6  commands_logic.cpp:
7  do_takeoff(), do_nav_wp(), do_land(), do_change_speed(), do_change_alt(),
8  do_vtol_transition(), do_nav_delay(), do_set_roi() ... ← 各种MAVLink任务命令
```

6.8 Log.cpp — 数据日志

```
1  日志消息类型：
2  CTUN ← 控制调参 (nav_roll/pitch, roll/pitch, throttle_out, airspeed)
3  NTUN ← 导航调参 (wp_distance, bearing, alt_error, crosstrack,
   airspeed_error)
4  QTUN ← 四旋翼调参 (quat, rate, accel targets)
5  AETR ← 执行器输出 (aileron/elevator/throttle/rudder 归一化值)
6  ATRP ← 姿态速率P
7  STAT ← 状态 (flight stage, failsafe, nav state...)
8  PIQR/PIQP/PIQY/PIQA ← 四旋翼 PIDs
9
10  关键函数：
11  Log_Write_Control_Tuning(), Log_Write_Nav_Tuning(),
12  Log_Write_Attitude(), Log_Write_AETR(),
   Log_Write_Vehicle_Startup_Messages()
```

6.9 RC_Channel_Plane.cpp — 辅助开关

```
1 do_aux_function() 分发的主要选项：
2   INVERTED, REVERSE_THROTTLE, AUTO, CIRCLE, LOITER, GUIDED, RTL, FBWA, QRTL,
3   Q_ASSIST, CROW_SELECT, TER_DISABLE, LANDING_FLARE, FW_AUTOTUNE,
4   TRIM_TO_CURRENT_SERVO_RC, SOARING, AIRMODE, FLAP_AUTO ...
```

6.10 defines.h — 关键常量

```
1 Failsafe 状态:   FAILSAFE_NONE=0, FAILSAFE_SHORT=1, FAILSAFE_LONG=2,
   FAILSAFE_GCS=3
2 Short FS 动作:   BESTGUESS/CIRCLE, CIRCLE, FBWA, DISABLED, FBWB
3 Long FS 动作:   CONTINUE, RTL, GLIDE, PARACHUTE, AUTO, AUTOLAND
4
5 Stick Mixing:   NONE, FBW, VTOL_YAW, FBW_NO_PITCH
6 UseReverseThrust: 按模式位掩码 (AUTO_ALWAYS, LOITER, RTL, CIRCLE, CRUISE,
   FBWB, GUIDED...)
7 FlightOptions:  17个选项位 (DIRECT_RUDDER_ONLY, CLIMB_BEFORE_TURN,
   ACRO_YAW_DAMPER...)
8
9 Crash Detection: DISABLED, DISARM
10 AirMode:        OFF, ON, ASSISTED_FLIGHT_ONLY
11 Crow Flap:      FLYINGWING, FULLSPAN, PROGRESSIVE_CROW
```

七、定位问题的快速导航表

你想定位的问题	先看哪个文件
飞机飞不稳/姿态抖动	Attitude.cpp → PID控制 + mode.h → 对应模式的 run()
航点/航线不对	navigation.cpp → 导航逻辑 + commands.cpp → 航点设定
舵机不动/油门异常	servos.cpp → 输出混控 + 抑制逻辑
遥控器不响应/失控	radio.cpp → RC输入 + events.cpp → 失控处理
模式切不过去	system.cpp → set_mode() + 对应 mode_*.cpp → _enter()
垂直起降过渡问题	quadplane.cpp → control_auto() + transition.h → 状态机
VTOL降落位置不准	quadplane.cpp → vtol_position_controller()
Q_ASSIST误触发	VTOL_Assist.cpp → 四个assist条件
参数不生效	Parameters.h/.cpp → 索引是否正确
日志缺数据	Log.cpp → Log_Write_* 系列
自动起飞/自动降落异常	mode_takeoff.cpp / mode_autoland.cpp
电池失控保护误触发	events.cpp → handle_battery_failsafe()
Fence误触发	fence.cpp + defines.h → FenceAutoEnable

你想定位的问题	先看哪个文件
Soaring逻辑问题	<code>soaring.cpp</code> + <code>mode_thermal.cpp</code>
TECS高度/速度控制问题	<code>Plane.cpp</code> → <code>update_alt()</code> + <code>TECS_controller</code>
L1横向导航问题	<code>navigation.cpp</code> → <code>L1_controller</code>

八、总结

ArduPlane 的架构核心是三层结构:

- 1. **调度器层** (`Plane.cpp` `scheduler_tasks[]`) — 确定所有任务的执行顺序和频率
- 2. **模式系统** (`mode.h` `Mode`基类 + 26个派生类) — 每种飞行模式封装了自己的导航和稳定逻辑
- 3. **VTOL子系统** (`quadplane.h` `QuadPlane`类) — 条件编译, 在固定翼基础上透明叠加垂直起降能力

调用追踪的通用路径:

```
1 scheduler_tasks[] → control_mode->update() → control_mode->run()
2                                     ↓
3             navigate() (10Hz) → TECS / L1 →
calc_nav_roll/pitch/throttle
4                                     ↓
5             stabilize_roll/pitch/yaw() (FAST_TASK) → 舵机PWM
```

如果是 QuadPlane VTOL 模式, 则:

```
1 control_mode->run() → quadplane.control_auto() / control_hover() /
vtol_position_controller()
```